

PersicoCRM

Developer Guide — راهنمای برنامه‌نویس

Next.js **15**Updated **2026-06-19**Version **2.0**

Multi-tenant SaaS

Prisma + MySQL

+۱۸♦

API Route

+۳۵

ماژول داشبورد

Λ

سرویس / زیردامنه

CSRF

Rate Limiting

SSE

Real-time

JWT

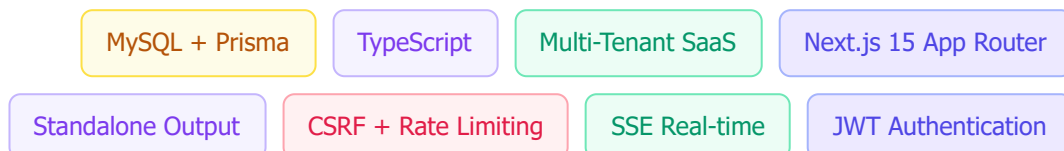
احراز هویت چندگانه

فهرست مطالب

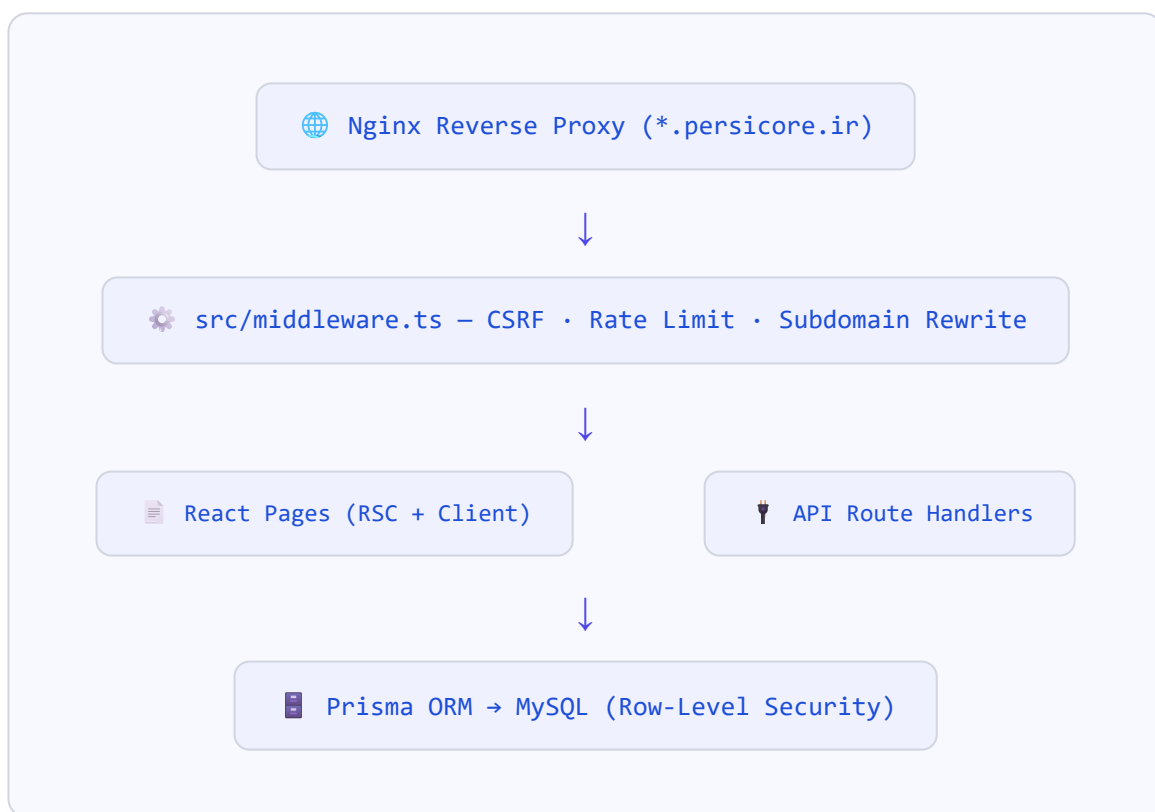
۱	معرفی کلی پروژه
۲	معماری سیستم
۲	زیرساخت فنی (Tech Stack)
۴	سرویس‌ها و زیردانه‌ها
۵	ساختار پوشه‌ها
۶	ماژول‌های داشبورد (+۳۵)
۷	پورتال مشتری
۸	پنل Super Admin
۹	ایجنت‌ساز AI
۱۰	محتواساز AI
۱۱	API Reference
۱۲	احراز هویت و امنیت
۱۲	پایگاه داده و Prisma
۱۴	متغیرهای محیطی (env.)
۱۵	الگوهای مهم کد
۱۶	راه‌اندازی توسعه
۱۷	استقرار (Production)

۱ معرفی کلی پروژه

PersicoCRM یک پلتفرم **SaaS چند-تانت** است که در یک اپلیکیشن Next.js 15 پیاده‌سازی شده و از طریق **subdomain routing** هشت سرویس مجزا ارائه می‌دهد. تمام سرویس‌ها از یک codebase و یک دیتابیس MySQL استفاده می‌کنند.



۲ معماری سیستم



نکته Multi-tenancy: هر tenant با `tenantId` در تمام جداول ایزوله است. در هر API call باید `tenantId` از JWT استخراج و در `WHERE clause` استفاده شود — در غیر این صورت داده‌های همه tenants برمی‌گردد.

زیرساخت فنی (Tech Stack)

۳

Frontend >

کتابخانه	نسخه	کاربرد
next	x.15	Framework — App Router + RSC + Standalone
react	x.19	UI Framework
typescript	x.5	Type Safety
tailwindcss	3.4	Utility-first CSS
shadcn/ui + radix-ui	latest	Accessible UI Components
@dnd-kit/core + sortable	/ 6.3 8.0	Drag & Drop — Kanban Boards
@tanstack/react-query	x.5	Server State / Data Fetching
@tanstack/react-table	x.8	Headless Advanced Tables
@tanstack/react-virtual	x.3	Virtualized Long Lists
recharts	x.2	Charts & Data Visualization
@tiptap/*	x.2	Rich Text Editor
motion (framer)	x.12	Animations
zustand	x.5	Client-side State Management
react-hook-form	x.7	Form Management
zod	x.3	Schema Validation
dayjs	x.1	Date/Time (Jalali support)
nuqs	x.2	URL Search Params State
cmdk	x.1	Command Palette (Ctrl+K)

کتابخانه	نسخه	کاربرد
<code>sonner</code>	x.1	Toast Notifications
<code>vaul</code>	x.1	Drawer Component
<code>fuse.js</code>	x.7	Client-side Fuzzy Search
<code>react-grid-layout</code>	1.5	Dashboard Grid Layout
<code>react-signature-canvas</code>	x.1	Digital Signature

Backend & Infrastructure ▸

ابزار	کاربرد
<code>@prisma/client 5.x</code>	Database ORM — MySQL
<code>@upstash/redis</code>	Redis Client (Edge + Node.js) — Rate Limiting
<code>jsonwebtoken</code>	JWT Generation & Verification
<code>bcryptjs</code>	Password Hashing
<code>@sentry/nextjs</code>	Error Tracking & Monitoring
<code>web-push (VAPID)</code>	Browser Push Notifications
<code>axios</code>	HTTP Client (برای external API calls)
<code>playwright</code>	E2E Testing

سرویس‌ها و زیردامنه‌ها ۲

`portal.persicore.ir`

پورتال اختصاصی مشتری

Route Group: (portal) — فاکتور، پروژه،
تیکت، پرداخت آنلاین

`crm.persicore.ir`

CRM اصلی + احراز هویت

Route Groups: (auth) + (dashboard) —
داشبورد کامل با ۳۵+ ماژول

`agent.persicore.ir`

ایجنت ساز AI

Route Group: (agent) — ساخت chatbot
بدون کد، دانش پایه، embed

`admin.persicore.ir`

پنل Super Admin

Route Group: (admin) — مدیریت tenantها،
پلن‌ها، پرداخت‌ها

`blog.persicore.ir`

بلاگ عمومی

Route Group: (blog) — مطالب، جستجو،
دسته‌بندی

`content.persicore.ir`

محتواساز AI

Route Group: (content) — تولید محتوا با
AI، تاریخچه، billing

`proposal.persicore.ir`

پروپوزال ساز

Route Group: (proposal) — نمایش عمومی
پروپوزال با slug

`resume.persicore.ir`

رزومه ساز

Route Group: (resume) — نمایش عمومی
رزومه با slug

نحوه کار Subdomain Routing

```
// src/middleware.ts — نمونه برای portal
if (hostname.startsWith("portal.")) {
  if (url.pathname.startsWith("/api/")) {
    url.pathname = "/api/portal" + url.pathname.slice(4);
  } else {
    url.pathname = "/portal" + url.pathname;
  }
  return NextResponse.rewrite(url);
}
```

```
}  
// همین الگو برای admin، resume، proposal، blog، content
```

ساختار پوشه‌ها

۵

```

crm/
├─ landing/                                # Next.js لندینگ استاتیک - جدا از
│   ├── index.html                        # persicore.ir صفحه اصلی
│   ├── styles.css                       # Aurora Glass طراحی
│   └─ script.js                         # CRM تم، انیمیشن، تب‌های
├─ prisma/
│   ├── schema.prisma                   # مدل‌های دیتابیس
│   ├── migrations/                    # Prisma migrations
│   └─ seed.ts                          # داده‌های اولیه
├─ src/
│   ├── app/
│   │   ├── (auth)/                     # ورود، ثبت‌نام، فراموشی رمز
│   │   ├── (dashboard)/                # اصلی - همه ماژول‌ها CRM
│   │   ├── (portal)/                  # پورتال مشتری
│   │   ├── (admin)/                   # Super Admin
│   │   ├── (agent)/                   # AI ایجنت‌ساز
│   │   ├── (content)/                 # AI محتواساز
│   │   ├── (blog)/                   # بلاگ عمومی
│   │   ├── (resume)/                 # رزومه عمومی
│   │   ├── (proposal)/                # پروپوزال عمومی
│   │   ├── (public)/                 # pricing، invoice-sign، contract-sign
│   │   └─ api/                       # API Route Handlers (180+ routes)
│   │       └─ api-docs/              # تعاملی API مستندات
│   ├── components/
│   │   ├── layout/                   # Sidebar، Topbar، CommandPalette
│   │   ├── ui/                      # shadcn/ui components
│   │   └─ common/                   # StatCard، EmptyState، ...
│   ├── lib/
│   │   ├── auth/                     # verifyAuth، JWT helpers
│   │   ├── db.ts                     # Prisma singleton
│   │   ├── ok.ts                     # ok() / err() helper
│   │   ├── redis.ts                  # @upstash/redis singleton (Edge + Node.js)
│   │   ├── rate-limit.ts             # Redis-backed rate limiter (API routes)
│   │   └─ utils/                     # formatPrice، toJalali، ...
│   ├── types/                       # TypeScript interfaces
│   └─ middleware.ts                  # CSRF + Rate Limit + Subdomain
└─ next.config.ts

```


 package.json .env.local

متغیرهای محیطی (ایجاد دستی)



ماژول‌های داشبورد (۲۵ + ماژول) ۶

CRM و فروش

مسیر	توضیح
/leads	مدیریت لیدها — Kanban + Table، Lead Scoring
/leads/deduplication	شناسایی و ادغام لیدهای تکراری
/pipeline	پایپلین فروش بصری
/clients & /clients/[id]	پروفایل ۳۶۰ درجه مشتری + تاریخچه تعاملات
/crm/segments	سگمنت‌بندی مشتریان
/crm/journey	Customer Journey Map
/crm/call-logs	لاگ تماس‌ها
/meetings	جلسات و قرار ملاقات‌ها
/sales-competition	رقابت فروش بین اعضای تیم
/commissions	مدیریت کمسیون‌ها

مالی و ERP

مسیر	توضیح
/invoicing & /invoicing/new	فاکتور با preview زنده + E-Signature
/erp/chart-of-accounts	نمودار حساب‌ها (دو طرفه)
/erp/journal-vouchers	سند حسابداری
/erp/ledger	دفتر کل
/erp/bank-accounts + reconciliation	بانکداری + مغایرت‌گیری
/erp/fixed-assets	دارایی‌های ثابت + استهلاک خودکار

مسیر	توضیح
/erp/budget	بودجه VS واقعی
/erp/tax	مدیریت مالیات
/erp/recurring-invoices	فاکتورهای دوره‌ای خودکار
/erp/reports	ترازنامه، سود/زیان، جریان نقدی، تراز آزمایشی

➤ منابع انسانی (HR)

مسیر	توضیح
/hr/attendance	حضور و غیاب
/hr/leave-requests	درخواست مرخصی
/hr/recruitment	استخدام و آنبوردینگ
/hr/kpi	ارزیابی عملکرد KPI
/hr/org-chart	چارت سازمانی تعاملی
/hr/shifts + overtime	شیفت‌ها و اضافه‌کاری
/payroll	حقوق و دستمزد + فیش حقوقی

➤ صنایع تخصصی

فروشگاه آنلاین /ecommerce/*	رستوران و کافه /restaurant/*	تولید و صنعت /manufacturing/*
آموزش و مدرسه /education/*	تجارت و واردات /trading/*	IT و نرم‌افزار /it/*
امور حقوقی /legal/*	مدیریت خدمات /service/*	انبارداری /inventory/*

➤ سایر ماژول‌های مهم

مسیر	توضیح
/marketing/*	کمپین، ایمیل، A/B Test، UTM، Persona، Analytics
/projects & /projects/[id]	پروژه‌ها + Milestone + Kanban Tasks
/contracts & /contracts/templates	قراردادها + قالب‌ها + E-Signature
/reports & /reports/builder	گزارش‌ساز + CSV export + Drag & Drop
/forms & /forms/builder	فرم‌ساز بصری
/messages	پیام‌رسان داخلی (SSE) Real-time
/settings/automation	موتور اتوماسیون If-This-Then-That
/settings/webhooks	Webhook به سیستم‌های خارجی
/timer	تایمر + Time Entries

پورتال مشتری ۷

دسترسی از `portal.persicore.ir` — سیستم JWT جداگانه (`portal_token`). هر مشتری (Client) می‌تواند با دعوتنامه وارد پورتال اختصاصی شرکت شود.

مسیر	توضیح
<code>/portal</code>	داشبورد مشتری — خلاصه وضعیت
<code>/portal/invoices</code>	مشاهده و پرداخت فاکتورها
<code>/portal/contracts</code>	قراردادها + امضای دیجیتال
<code>/portal/projects</code>	پیگیری پروژه‌های جاری
<code>/portal/tickets & /[id]</code>	تیکت پشتیبانی + پاسخ‌دهی
<code>/portal/files</code>	دانلود فایل‌ها و مستندات
<code>/portal/designs</code>	بررسی و تأیید طرح‌های گرافیکی
<code>/portal/messages</code>	مکالمه مستقیم با تیم
<code>/portal/nps</code>	نظرسنجی رضایت مشتری
<code>/portal/payment/callback</code>	بازگشت از درگاه پرداخت

API ها: تمام API های پورتال از پیشوند `/api/portal/` استفاده می‌کنند و از CSRF exempt هستند (سیستم auth جداگانه).

پنل Super Admin

دسترسی از `admin.persicore.ir` — فقط برای مدیران سیستم. JWT جداگانه (`admin_token`).

پلن‌های اشتراک <code>/admin/plans</code>	مدیریت کاربران <code>/admin/users</code>	مدیریت Tenantها <code>/admin/tenants</code>
مدیریت بلاگ <code>/admin/blog</code>	مالی کلی <code>/admin/finance-overview</code>	پرداخت‌ها <code>/admin/payments</code>
ایجنت‌ها <code>/admin/agents</code>	پروپوزال‌ها <code>/admin/proposals</code>	رزومه‌سازها <code>/admin/resumes</code>
آنالیز مصرف <code>/admin/usage</code>	کد تخفیف <code>/admin/coupons</code>	Content AI <code>/admin/content</code>

ایجنت ساز AI ۹

دسترسی از `agent.persicore.ir` — ثبت نام مستقل با موبایل + OTP. هر کاربر می تواند چندین ايجنت بسازد، آموزش دهد و در سایت embed کند.

مسیر	توضیح
<code>/agent/register & /login</code>	ثبت نام / ورود با OTP
<code>/agent/dashboard</code>	لیست ايجنت های ساخته شده
<code>/agent/new</code>	ایجاد ايجنت جدید
<code>/agent/plans</code>	پلن های قیمت گذاری
<code>/agent/agents/[id]</code>	تنظیمات اصلی ايجنت
<code>/agent/agents/[id]/knowledge</code>	دانش پایه — URL crawler + آپلود فایل
<code>/agent/agents/[id]/customize</code>	شخصی سازی ظاهر (رنگ، آواتار، نام)
<code>/agent/agents/[id]/embed</code>	کد JavaScript برای embed در سایت
<code>/agent/agents/[id]/conversations</code>	تاریخچه کامل مکالمات

API های کلیدی ايجنت

POST /api/agent/auth/register	— ثبت نام
POST /api/agent/auth/send-otp	— ارسال OTP
POST /api/agent/auth/verify-otp	— تأیید OTP
GET /api/agent/agents	— لیست ايجنت ها
POST /api/agent/agents	— ایجاد ايجنت
GET /api/agent/agents/[id]	— جزئیات ايجنت
PUT /api/agent/agents/[id]/customization	— ظاهر
POST /api/agent/agents/[id]/knowledge	— اضافه کردن دانش
POST /api/agent/agents/[id]/knowledge/crawl	— Crawl URL
GET /api/agent/chat/[agentId]	— Chat (SSE streaming)
POST /api/agent/upload	— آپلود فایل به دانش
GET /api/agent/wordpress-plugin	— اطلاعات پلاگین WP

محتواساز AI ۱۰

دسترسی از content.persicore.ir — تولید محتوای سئو محور با AI برای بلاگ، شبکه‌های اجتماعی و ایمیل مارکتینگ.

توضیح	API
ثبت‌نام، ورود، OTP	/api/content/auth/*
تولید محتوا با AI	/api/content/generate
پیشنهاد زیرموضوع	/api/content/subtopics
تاریخچه محتواهای تولیدشده	/api/content/history
مدیریت اشتراک + پرداخت	/api/content/billing/*

دسته‌بندی کلی — API Reference

۱۱

➤ احراز هویت CRM

POST /api/auth/send-otp	POST /api/auth/verify-otp
POST /api/auth/login	POST /api/auth/register
POST /api/auth/logout	GET /api/auth/me
POST /api/auth/forgot-password	POST /api/auth/reset-password
POST /api/auth/change-password	POST /api/auth/2fa/setup
POST /api/auth/2fa/verify	

➤ Push و Real-time

GET /api/stream	— real-time رویدادهای — SSE
GET /api/push/vapid-key	— VAPID Public Key
POST /api/push/subscribe	— Push Subscription ثبت
GET /api/portal/stream	— SSE پورتال مشتری

➤ فایل‌ها

POST /api/upload	— آپلود فایل
GET /api/files/[...path]	— دریافت فایل
CRUD /api/file-items	— متادیتا فایل‌ها
GET /api/file-folders	— پوشه‌ها

➤ امضای دیجیتال (Public — بدون Auth)

GET /api/invoices/sign/[token]	— اطلاعات فاکتور برای امضا
POST /api/invoices/sign/[token]	— ثبت امضا
GET /api/contracts/sign/[token]	— اطلاعات قرارداد
POST /api/contracts/sign/[token]	— ثبت امضا

➤ گزارشات ERP

GET /api/erp/reports/balance-sheet	— ترازنامه
GET /api/erp/reports/profit-loss	— سود و زیان

GET /api/erp/reports/cash-flow	جریان نقدی -
GET /api/erp/reports/trial-balance	تراز آزمایشی -
GET /api/erp/reports/general-ledger	دفتر کل -
GET /api/erp/reports/aged-receivables	سنی دریافتنی -
GET /api/erp/reports/aged-payables	سنی پرداختنی -

Automation و Webhooks ▸

CRUD /api/webhooks	webhooks مدیریت -
POST /api/webhooks/[id]/test	webhook تست -
CRUD /api/automation	موتور اتوماسیون -

مستندات تعاملی: مسیر [api-docs/](#) در CRM داشبورد موجود است.

احراز هویت و امنیت ۱۲

جریان احراز هویت

1. کاربر موبایل وارد می‌کند
2. POST /api/auth/send-otp → SMS ارسال
3. POST /api/auth/verify-otp → تأیید کد
4. POST /api/auth/login → دریافت JWT
5. JWT در httpOnly cookie ذخیره می‌شود: auth_token
6. استخراج می‌شود tenantId → کوکی را بررسی می‌کند API call هر

ساختار JWT Payload

```
// CRM Token
{
  userId: string,
  tenantId: string,
  role: "owner" | "admin" | "manager" | "user",
  iat: number,
  exp: number
}
```

سیستم‌های JWT مجزا

توضیح	Cookie Name	سرویس
برای dashboard و API های اصلی	auth_token	CRM Dashboard
برای مشتریان پورتال	portal_token	Client Portal
فقط Super Admin	admin_token	Super Admin
کاربران ایجنت‌ساز	agent_token	Agent Builder
کاربران محتواساز	content_token	Content AI

Rate Limiting — Redis-backed و CSRF

```
// CSRF Protection (src/middleware.ts)
- معاف هستند (GET, HEAD, OPTIONS) روش‌های ایمن
- Exempt: /api/auth/, /api/portal/, /api/agent/, /api/content/
- بررسی Origin header vs Host header

// Rate Limiting – Redis-backed (multi-instance safe)
- Authenticated: 100 req/min ← per tenantId + path
- Anonymous: 30 req/min ← per IP + path
- Exempt: /api/auth/, /api/agent/, /api/stream, /api/files
- الگوریتم: Lua script اتمیک – INCR + PEXPIRE (بدون race condition)
- Fail-open: باشد، درخواست رد نمی‌شود Redis down اگر
```

➤ معماری دو لایه Rate Limiting

لایه	فایل	Runtime	کاربرد
Middleware	src/middleware.ts	Edge	عمومی — همه api routes
Route Handler	src/lib/rate-limit.ts	Node.js	اختصاصی — auth, OTP, register

```
// src/lib/redis.ts – singleton client (Edge + Node.js)
import { Redis } from "@upstash/redis";

let _redis: Redis | null = null;
export function getRedis(): Redis {
  if (!_redis) {
    _redis = new Redis({
      url: process.env.UPSTASH_REDIS_REST_URL!,
      token: process.env.UPSTASH_REDIS_REST_TOKEN!,
    });
  }
  return _redis;
}

// Lua script به Redis round-trip اتمیک – یک
const SCRIPT = `
  local c = redis.call('INCR', KEYS[1])
  if c == 1 then redis.call('PEXPIRE', KEYS[1], tonumber(ARGV[1])) end
  return {c, redis.call('PTTL', KEYS[1])}
`

// API routes استفاده در
```

```
const rl = await rateLimit(`crm-login:${ip}`, 10, 15 * 60 * 1000);  
if (!rl.allowed) return err("429", "نرخ درخواست بیش از حد مجاز");
```

✅ **Redis راه‌اندازی شده:** Rate limiting در هر دو لایه (middleware + API routes) از upstash/redis@ استفاده می‌کند. برای راه‌اندازی به بخش ۱۶ مراجعه کنید.

2FA (Two-Factor Authentication) ▸

TOTP-based (سازگار با Google Authenticator):

POST /api/auth/2fa/setup — QR Code راه‌اندازی و دریافت
POST /api/auth/2fa/verify — تأیید کد

پایگاه داده و Prisma

۱۳

دستورات مهم

```
npm run db:migrate    # migrations موجود (production) اجرای
npx prisma migrate dev --name feature_name # migration جدید (dev)
npm run db:studio     # Prisma Studio – GUI در مرورگر دیتابیس
npm run db:seed       # Seed داده‌های اولیه
```

الگوی Multi-tenancy در Prisma

هر مدل اصلی دارای `tenantId` است:

```
// prisma/schema.prisma
model Lead {
  id          String @id @default(cuid())
  tenantId    String
  name        String
  email       String?
  // ...
  tenant      Tenant @relation(fields: [tenantId], references: [id])
}
```

استفاده صحیح در API

```
// ✅ الزامی - WHERE در tenantId همیشه
const { tenantId } = await verifyAuth(request);
const leads = await prisma.lead.findMany({
  where: { tenantId }, // فراموش نشود ←
  include: { client: true },
  orderBy: { createdAt: "desc" },
});

// ❌ tenantId بدون - همه tenantId ها برمی‌گرداند
const leads = await prisma.lead.findMany(); // WRONG
```

Prisma Singleton

```
// src/lib/db.ts
import { PrismaClient } from "@prisma/client";
const g = globalThis as { prisma?: PrismaClient };
export const prisma = g.prisma ?? new PrismaClient();
if (process.env.NODE_ENV !== "production") g.prisma = prisma;
```

متغیرهای محیطی (env.local.)

۱۲

```
# — Database —————
DATABASE_URL="mysql://user:password@localhost:3306/persicore_crm"

# — Redis (Rate Limiting – multi-instance safe) —————
# گزینه ۱: Upstash Cloud (رایگان): https://console.upstash.com
# گزینه ۲: Self-hosted با HTTP proxy:
# docker run -p 8079:8079 -e SRH_TOKEN=mysecret \
#   -e SRH_CONNECTION_STRING="redis://localhost:6379" \
#   hiett/serverless-redis-http:latest
UPSTASH_REDIS_REST_URL="https://your-instance.upstash.io"
UPSTASH_REDIS_REST_TOKEN="your-token-here"

# — JWT Secrets (هر سرویس جداگانه) —————
JWT_SECRET="your-crm-secret-min-32-chars"
ADMIN_JWT_SECRET="your-admin-secret"
AGENT_JWT_SECRET="your-agent-secret"
CONTENT_JWT_SECRET="your-content-secret"
JWT_EXPIRES_IN="7d"

# — App URL —————
NEXT_PUBLIC_APP_URL="https://crm.persicore.ir"

# — Sentry —————
SENTRY_DSN="https://xxx@sentry.io/project"
NEXT_PUBLIC_SENTRY_DSN="https://xxx@sentry.io/project"
SENTRY_ORG="your-org"
SENTRY_PROJECT="persicore-crm"

# — File Storage —————
STORAGE_TYPE="local"          # یا "s3"
UPLOAD_DIR="./uploads"        # برای local
# برای S3 / MinIO:
S3_ENDPOINT="https://s3.example.com"
S3_BUCKET="persicore-files"
S3_ACCESS_KEY="..."
S3_SECRET_KEY="..."
NEXT_PUBLIC_CDN_URL="https://cdn.persicore.ir"

# — SMS (OTP) —————
```



```
SMS_PROVIDER="kavenegar"      # یا گزینه های دیگر
SMS_API_KEY="your-sms-key"

# — Web Push Notifications —————
VAPID_PUBLIC_KEY="BNxxx..."
VAPID_PRIVATE_KEY="xxx..."
VAPID_EMAIL="admin@persicore.ir"

# — AI (Agent Builder + Content AI) —————
OPENAI_API_KEY="sk-proj-..."

# — Payment (Zarrinpal) —————
ZARINPAL_MERCHANT_ID="xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxxx"
ZARINPAL_SANDBOX="false"      # true در development

# — CRON Secret —————
CRON_SECRET="your-cron-secret"
```

⚠️ **امنیت:** فایل `env.local` هرگز به `git commit` نشود. مطمئن شوید در `gitignore` قرار دارد.

الگوهای مهم کد ۱۵

الگوی API Response >

```
// src/lib/ok.ts
export function ok(data: unknown, status = 200) {
  return NextResponse.json({ success: true, data }, { status });
}

export function err(message: string, status = 400) {
  return NextResponse.json({ success: false, error: message }, { status });
}

// Route Handler استفاده در
export async function GET(request: NextRequest) {
  const { tenantId } = await verifyAuth(request);
  const clients = await prisma.client.findMany({ where: { tenantId } });
  return ok(clients);
}

// Client Component استفاده در
const res = await fetch("/api/clients");
const json = await res.json();
const clients = json.data; // ← r.data.data pattern
```

Promise به صورت Next.js 15 — params >

```
// ✅ در Next.js 15 صحیح
export default async function ProjectPage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  const { id } = await params; // ← باید await شود
  const project = await fetchProject(id);
  return <div>{project.name}</div>;
}

// ❌ در Next.js 14 اشتباه - کار می‌کرد اما 15 خیر
export default async function Page({ params }: { params: { id: string } }) {
```

```
const { id } = params; // WRONG in Next.js 15
}
```

SSE Real-time ▸

```
// Server: /api/stream/route.ts
export async function GET(req: NextRequest) {
  const { tenantId } = await verifyAuth(req);
  const encoder = new TextEncoder();
  let intervalId: NodeJS.Timeout;

  const stream = new ReadableStream({
    start(controller) {
      const send = (event: string, data: object) =>
        controller.enqueue(
          encoder.encode(`event: ${event}\ndata: ${JSON.stringify(data)}\n\n`
        );
      intervalId = setInterval(() => send("ping", { t: Date.now() }), 30000);
    },
    cancel() { clearInterval(intervalId); },
  });

  return new Response(stream, {
    headers: {
      "Content-Type": "text/event-stream",
      "Cache-Control": "no-cache, no-transform",
      "Connection": "keep-alive",
    },
  });
}

// Client
const es = new EventSource("/api/stream");
es.addEventListener("message_created", (e) => {
  const data = JSON.parse(e.data);
  // ...
});
```

API الگوی Multi-tenant در ▸

```
// باید این الگو را رعایت کند Route Handler هر
export async function POST(request: NextRequest) {
  // 1. استخراج و احراز هویت tenantId
  const { tenantId, userId } = await verifyAuth(request);

  // 2. validation ورودی
  const body = await request.json();
  const parsed = CreateLeadSchema.safeParse(body);
  if (!parsed.success) return err("422", "نامعتبر");

  // 3. tenantId عملیات دیتابیس همیشه با
  const lead = await prisma.lead.create({
    data: { ...parsed.data, tenantId, createdById: userId },
  });

  return ok(lead, 201);
}
```

راہ اندازی توسعه ۱۶

پیش نیازها

Redis (یا Upstash)

+MySQL 8

npm / pnpm

+Node.js 18

راہ اندازی Redis

```
# (سریع ترین، بدون نصب) Upstash Cloud - گزینه ۱:
# → https://console.upstash.com → Create Database → REST API
# قرار دهید env.local را در Token و URL

# (نصب شده Redis نیاز به Docker با Self-hosted - گزینه ۲):
docker run -d -p 8079:8079 \
  -e SRH_MODE=env \
  -e SRH_TOKEN=my-dev-secret \
  -e SRH_CONNECTION_STRING="redis://host.docker.internal:6379" \
  hiett/serverless-redis-http:latest

# در env.local:
UPSTASH_REDIS_REST_URL=http://localhost:8079
UPSTASH_REDIS_REST_TOKEN=my-dev-secret

# (غیرفعال rate limiting) می شود Redis، middleware fail-open نکته: بدون
```

نصب و راه اندازی

```
# ۱. dependencies نصب
cd crm
npm install # یا: pnpm install

# ۲. و پر کردن مقادیر env.local. ایجاد فایل
cp .env.example .env.local

# ۳. راه اندازی دیتابیس
npm run db:migrate # migrations اجرای همه
npm run db:seed # داده های اولیه (اختیاری)
```

اجرای سرور توسعه ۴۰

npm run dev

→ http://localhost:3000

دسترسی به سرویس‌ها در Development

URL در localhost	سرویس
http://localhost:3000/login	CRM Dashboard
http://localhost:3000/portal	Client Portal
http://localhost:3000/admin	Super Admin
http://localhost:3000/agent	Agent Builder
http://localhost:3000/content	Content AI
http://localhost:3000/blog	Blog
http://localhost:3000/pricing	Public Pricing
http://localhost:3000/api-docs	API Docs

دستورات مفید

```

npm run dev          # Development server
npm run build         # Production build (prisma generate + next build)
npm start            # اجرای standalone server
npm run lint         # ESLint check
npm run db:migrate   # اجرای migrations
npm run db:studio    # Prisma Studio GUI
npm run db:seed      # Seed دیتابیس
npm run test:e2e     # E2E tests با Playwright
npm run test:e2e:ui  # Playwright با UI

```

استقرار (Production) ۱۷

Build ▶

```
# Build
npm run build
# معادل: prisma generate && next build

# Output: .next/standalone/ (فایل‌های قابل استقرار)
```

Docker ▶

```
FROM node:18-alpine AS builder
WORKDIR /app
COPY . .
RUN corepack enable && pnpm install --frozen-lockfile
RUN pnpm run build

FROM node:18-alpine AS runner
WORKDIR /app
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./next/static
COPY --from=builder /app/public ./public
EXPOSE 3000
ENV NODE_ENV=production PORT=3000 HOSTNAME=0.0.0.0
CMD ["node", "server.js"]
```

Port همه Subdomain به یک Nginx ▶

```
server {
    listen 443 ssl;
    server_name ~^(.+)\.persicore\.ir$ persicore.ir;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
```

```

    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_cache_bypass $http_upgrade;
}

# SSE - غیرفعال کردن buffering
location /api/stream {
    proxy_pass http://127.0.0.1:3000;
    proxy_buffering off;
    proxy_cache off;
    proxy_set_header Connection '';
    proxy_http_version 1.1;
    chunked_transfer_encoding on;
}
}

```

چک لیست Pre-deploy ▶

- همه متغیرهای محیطی در `env.production.` تنظیم شده‌اند
- تمام JWT Secret ها حداقل ۳۲ کاراکتر تصادفی هستند
- `ZARINPAL_SANDBOX=false` تنظیم شده است
- روی دیتابیس production اجرا شده است `npm run db:migrate`
- Sentry تنظیم شده و پروژه در Sentry ایجاد شده است
- Storage به S3/MinIO migration شده است (برای multi-instance)
- `UPSTASH_REDIS_REST_URL` و `TOKEN` تنظیم شده‌اند (rate limiting Redis-backed)
- SSL Certificate برای همه subdomain ها تنظیم شده است